

13. Facade Design Pattern in Java

13.1 Introduction

The **Facade Design Pattern** is a **structural pattern** that provides a simplified interface to a complex subsystem.

13.2 Problem it Solves

When a system becomes complex with many moving parts, it becomes hard to use or integrate. Facade hides the complexity and exposes only the necessary functionality.

13.3 What is Decorator Pattern?

Facade is a single class that provides a unified and easy-to-use interface to a set of interfaces in a subsystem.

13.4 Real-World Analogy

When using a remote control to operate a TV, you don't need to know how the internal circuits work. The remote control is a **facade**.

13.5 Structure

```
package com.mannemlokes;

// Complex Subsystems
class Engine {
    void start() {
        // ...
    }
}

class FuelPump {
    void pump() {
        // ...
    }
}

class Battery {
    void supplyPower() {
        // ...
    }
}

// Facade
class CarFacade {
    private Engine engine = new Engine();
    private FuelPump pump = new FuelPump();
    private Battery battery = new Battery();

    public void startCar() {
        battery.supplyPower();
        pump.pump();
        engine.start();
    }
}
```

```
}
```

13.6 Benefits

- Simplifies client interaction with complex systems
- Reduces dependencies between subsystems and clients
- Promotes loose coupling

13.7 Implementation Steps

1. Identify a complex subsystem
2. Create a facade class
3. Delegate requests to appropriate subsystem classes

13.8 Examples

Example1: Computer Start-Up

```
package com.mannemlokesesh;  
  
class CPU {  
    public void freeze() {  
        System.out.println("Freezing CPU");  
    }  
  
    public void jump(long position) {  
        System.out.println("Jumping to: " + position);  
    }  
  
    public void execute() {  
        System.out.println("Executing");  
    }  
}  
  
class Memory {  
    public void load(long position, String data) {  
        System.out.println("Loading data to memory at position: " +  
                             position);  
    }  
}  
  
class HardDrive {  
    public String read(long lba, int size) {  
        return "OS Boot Data";  
    }  
}  
  
class ComputerFacade {  
    private CPU cpu;  
    private Memory memory;  
    private HardDrive hardDrive;  
  
    public ComputerFacade() {  
        this.cpu = new CPU();  
        this.memory = new Memory();  
        this.hardDrive = new HardDrive();  
    }  
}
```

```
        public void start() {
            cpu.freeze();
            memory.load(0, hardDrive.read(100, 50));
            cpu.jump(0);
            cpu.execute();
        }
    }

    public class Example1FacadeTest {
        public static void main(String[] args) {
            ComputerFacade computer = new ComputerFacade();
            computer.start();
        }
    }
}
```

Output:

```
Freezing CPU
Loading data to memory at position: 0
Jumping to: 0
Executing
```

Example2: Travel Booking System

```
package com.mannemlokes;

class FlightBooking {
    public void bookFlight() {
        System.out.println("Flight booked");
    }
}

class HotelBooking {
    public void bookHotel() {
        System.out.println("Hotel booked");
    }
}

class CabBooking {
    public void bookCab() {
        System.out.println("Cab booked");
    }
}

class TravelFacade {
    private FlightBooking flight;
    private HotelBooking hotel;
    private CabBooking cab;

    public TravelFacade() {
        flight = new FlightBooking();
        hotel = new HotelBooking();
        cab = new CabBooking();
    }

    public void bookTravelPackage() {
        flight.bookFlight();
        hotel.bookHotel();
    }
}
```

```

        cab.bookCab();
    }
}

public class Example2TravelApp {
    public static void main(String[] args) {
        TravelFacade travel = new TravelFacade();
        travel.bookTravelPackage();
    }
}

```

Output:

```

Flight booked
Hotel booked
Cab booked

```

Example3: E-Commerce Order Processing

```

package com.mannemlokes;

class InventoryService {
    public boolean checkStock(String item) {
        System.out.println("Checking stock for: " + item);
        return true;
    }
}

class PaymentService {
    public void makePayment(String account, double amount) {
        System.out.println("Payment of $" + amount + " made by " + account);
    }
}

class ShippingService {
    public void shipProduct(String item) {
        System.out.println("Shipping: " + item);
    }
}

class OrderFacade {
    private InventoryService inventory = new InventoryService();
    private PaymentService payment = new PaymentService();
    private ShippingService shipping = new ShippingService();

    public void placeOrder(String item, String account, double amount) {
        if (inventory.checkStock(item)) {
            payment.makePayment(account, amount);
            shipping.shipProduct(item);
        }
    }
}

public class Example3ECommerce {
    public static void main(String[] args) {
        OrderFacade order = new OrderFacade();
        order.placeOrder("iPhone 15", "user123", 999.99);
    }
}

```

}

Output:

```
Checking stock for: iPhone 15
Payment of $999.99 made by user123
Shipping: iPhone 15
```

13.9 Facade vs Other Patterns

Pattern	Purpose
Facade	Simplify access to a complex subsystem
Mediator	Coordinate communication between objects
Adapter	Convert one interface to another
Builder	Step-by-step object creation

13.10 Pros and Cons

**Pros**

- Simplifies usage of complex APIs
- Reduces coupling
- Promotes clean architecture

**Cons**

- Facade may become a God object
- Hides internal subsystem logic (may reduce flexibility)

13.11 When to Use / Not to Use

Use When:

- You want to simplify interactions with complex subsystems
- You want to decouple clients from subsystems

Avoid When:

- Subsystem is already simple
- You need full control over each subsystem

13.12 Summary

Feature	Description
Intent	Provide simplified access to subsystem
Use Case	APIs, startup procedures, aggregators
Key Benefit	Unified, clean interface for complex logic

The **Facade Pattern** is ideal when you want to abstract and simplify a set of operations or services into a single high-level API.